

KuiklyWebView JSBridge 技术文档

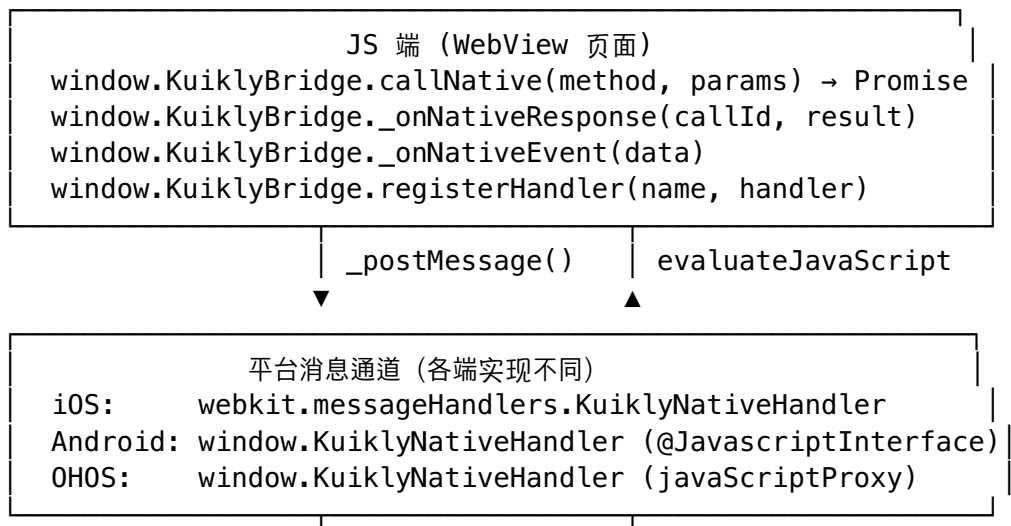
本文档基于 KuiklyWebview 项目源码分析，详细记录 WebView 与 Native 之间的 JSBridge 通信机制。覆盖 iOS、Android、OHOS/鸿蒙 三端实现。

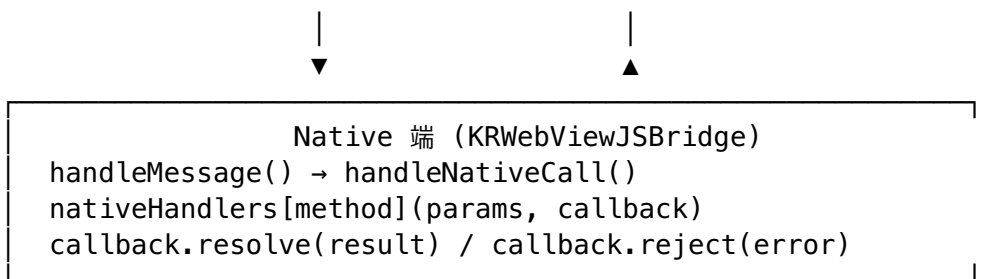
目录

- 1. 架构概览
- 2. 核心原理：WebView 执行 JS 字符串
- 3. Kotlin 共享层协议定义
- 4. 三端原生 WebView 实现对比
- 5. JS → Native 通信机制
- 6. Native → JS 通信机制
- 7. Native 对象如何挂载到 JS 环境
- 8. Native 处理 JSON 消息的流程
- 9. 消息路由：从 postMessage 到 handleMessage
- 10. Android @JavascriptInterface 反射机制详解
- 11. 完整调用链路总结

1. 架构概览

KuiklyWebView 的 JSBridge 采用 消息传递 + Promise 回调 的设计模式，通过 Kotlin Multiplatform 共享层统一协议定义，三端各自实现平台特有的通信通道。





关键源文件

文件	作用
KuiklyWebview/src/commonMain/ kotlin/.../JSBridgeProtocol.kt	Kotlin 共享层，定义协议常量、JS 注入脚本、工具方法
KuiklyWebviewAndroid/.../ KRWebViewJSBridge.kt	Android 端 JSBridge 实现
KuiklyWebviewAndroid/.../ KRWebView.kt	Android 端 WebView 封装（挂载 JSBridge）
KuiklyWebviewIOS/Classes/ KRWebViewJSBridge.m/.h	iOS 端 JSBridge 实现
KuiklyWebviewIOS/Classes/ KRWebView.m	iOS 端 WebView 封装（挂载 JSBridge）
KuiklyWebview0hos/.../ KRWebViewJSBridge.ets	OHOS/鸿蒙端 JSBridge 实现
KuiklyWebview0hos/.../ KRWebView.ets	OHOS/鸿蒙端 WebView 封装（挂载 JSBridge）

2. 核心原理：WebView 执行 JS 字符串

所有平台的原生 WebView 都支持将 JS 代码以字符串形式传入并执行。这是 JSBridge 工作的基础。

平台	Native → JS（执行 JS 字符串）	API
iOS	WKWebView	evaluateJavaScript(_:completionHandler:)
Android	android.webkit.WebView	evaluateJavascript(script, callback)
OHOS	ArkWeb WebviewController	runJavaScript(script)

使用场景包括：- 注入 Bridge 脚本：页面加载时注入 BRIDGE_JS_CODE + 平台 _postMessage 实现 - 发送事件到 JS：构建 _onNativeEvent(...) 脚本字符串后执行 - 回调 JS Promise：构建 _onNativeResponse(callId, result, error) 脚本后执行

3. Kotlin 共享层协议定义

文件：JSBridgeProtocol.kt

3.1 协议常量

```
const val BRIDGE_NAME =
    "KuiklyBridge" // JS 端 Bridge 对象名
const val NATIVE_HANDLER_NAME = "KuiklyNativeHandler" // Native 消
    息处理器名

// 消息 JSON Key
const val KEY_CALL_ID = "callId"
const val KEY_METHOD = "method"
const val KEY_PARAMS = "params"
const val KEY_TYPE = "type"

// 消息类型
const val TYPE_CALL = "call" // JS 调用 Native
const val TYPE_RESPONSE = "response" // Native 返回结果
const val TYPE_EVENT = "event" // Native 主动推送
```

3.2 Bridge JS 核心脚本 (BRIDGE_JS_CODE)

注入后在 window.KuiklyBridge 上提供以下能力：

方法	作用
callNative(method, params)	JS 调用 Native, 返回 Promise
_onNativeResponse(callId, result, error)	Native 回调 Promise
_onNativeEvent(data)	Native 主动推送事件
registerHandler(name, handler)	JS 端注册事件处理器
_postMessage(message)	占位方法，由各平台脚本覆盖

3.3 平台 _postMessage 覆盖脚本

平台	常量名	JS 实际调用
iOS	IOS_POST_MESSAGE_JS	window.webkit.messageHandlers.KuiklyNativeHandler
Android	ANDROID_POST_MESSAGE_JS	window.KuiklyNativeHandler.postMessage(message)
OHOS	OHOS_POST_MESSAGE_JS	window.KuiklyNativeHandler.postMessage(message)

策略模式应用：Bridge 核心逻辑统一，只有 _postMessage 一个方法根据平台不同而不同。

3.4 工具方法

- **buildResponseScript(callId, result, error)** : 构建 Native 回调 JS Promise 的脚本
 - **buildEventScript(method, params)** : 构建 Native 主动推送事件的脚本
 - **escapeJsString(str)** : O(n) 单次遍历转义 JS 字符串特殊字符, 覆盖反斜杠、引号、换行符、Unicode 行/段分隔符、null 字符、</script> 标签
-

4. 三端原生 WebView 实现对比

iOS 端 (WKWebView)

```
// 注入 Bridge 脚本
[self.webView evaluateJavaScript:[KRWebViewJSBridge
cachedFullBridgeScript] completionHandler:nil];

// 发送事件到 JS
[self.webView evaluateJavaScript:script completionHandler:nil];
```

- JS → Native 通道 : WKScriptMessageHandler 协议
- 预缓存脚本 : 通过 dispatch_once 确保只拼接一次

Android 端 (android.webkit.WebView)

```
// 注入 Bridge 脚本
webView.evaluateJavascript(CACHED_FULL_BRIDGE_SCRIPT, null)

// 发送事件到 JS
webView.evaluateJavascript(script, null)
```

- JS → Native 通道 : @JavascriptInterface 注解
- 预缓存脚本 : 通过 by lazy 延迟初始化

OHOS/鸿蒙端 (ArkWeb)

```
// 注入 Bridge 脚本
this.controller.runJavaScript(CACHED_BRIDGE_SCRIPT);

// 发送事件到 JS
this.controller.runJavaScript(script);
```

- JS → Native 通道 : javascriptProxy 机制
 - 通过 methodList 显式声明暴露的方法
-

5. JS → Native 通信机制

5.1 JS 端发起调用

```
// JS 业务代码
const result = await
  window.KuiklyBridge.callNative('getDeviceInfo', { key:
    'model' });
```

5.2 callNative 内部流程

```
callNative: function(method, params) {
  return new Promise(function(resolve, reject) {
    var id = 'cb_' + (_callId++); //
    ① 生成唯一 callId
    _callbacks[id] = { resolve: resolve, reject: reject }; //
    ② 保存 Promise 回调
    var timer = setTimeout(function() {
      cleanupCallback(id);
    }, CALLBACK_TIMEOUT); //
    ③ 30秒超时保护
    _callbacks[id]._timer = timer;
    var message = JSON.stringify({ //
      ④ 序列化为 JSON
      type: 'call',
      callId: id,
      method: method,
      params: params || {}
    });
    window.KuiklyBridge._postMessage(message); //
    ⑤ 发送到 Native
  });
}
```

5.3 消息 JSON 格式

```
{
  "type": "call",
  "callId": "cb_0",
  "method": "getDeviceInfo",
  "params": { "key": "model" }
}
```

5.4 _postMessage 三端路由

```
JS: _postMessage(jsonString)
  |— iOS:
window.webkit.messageHandlers.KuiklyNativeHandler.postMessage(msg)
  |— Android: window.KuiklyNativeHandler.postMessage(msg)
  [@JavascriptInterface]
```

```
└─ OHOS:    window.KuiklyNativeHandler.postMessage(msg)
[javaScriptProxy]
```

6. Native → JS 通信机制

6.1 回调 JS Promise

Native 处理完调用后，构建回调脚本：

```
// Kotlin 共享层
fun buildResponseScript(callId: String, result: String?, error:
    String?): String {
    val resultStr = result ?: "null"
    val errorStr = if (error != null) "'$
        {escapeJsString(error)}'" else "null"
    return "window.KuiklyBridge._onNativeResponse('$
        {escapeJsString(callId)}', $resultStr, $errorStr);"
}
```

生成的 JS 脚本示例：

```
window.KuiklyBridge._onNativeResponse('cb_0', {"model":"iPhone
    15"}, null);
```

6.2 JS 端 _onNativeResponse 处理

```
_onNativeResponse: function(callId, result, error) {
    var cb = _callbacks[callId];          // 通过 callId 找到 Promise
    if (cb) {
        if (cb._timer) clearTimeout(cb._timer); // 清除超时计时器
        if (error) {
            cb.reject(new Error(error));          // 有错误 →
            reject
        } else {
            cb.resolve(result);                    // 无错误 →
            resolve
        }
        delete _callbacks[callId];              // 清理，防止内存
        泄漏
    }
}
```

6.3 Native 主动推送事件

```
fun buildEventScript(method: String, params: String): String {
    return "window.KuiklyBridge._onNativeEvent({method:'$
        {escapeJsString(method)}',params:$params});"
}
```

JS 端通过 registerHandler 注册事件处理器来接收。

7. Native 对象如何挂载到 JS 环境

JS 中的 window.KuiklyNativeHandler 不是 JS 自己创建的，而是各平台原生层在 **WebView** 初始化时预先注入 的。

7.1 iOS — addScriptMessageHandler

```
// KRWebView.m - setupWebView 中
[self.webView.configuration.userContentController
    addScriptMessageHandler:self.jsBridge
    name:@"KuiklyNativeHandler"];
```

- WKWebView 自动在 JS 环境中创建 window.webkit.messageHandlers.KuiklyNativeHandler
- 该对象自带 postMessage() 方法
- JS 调用时触发 didReceiveScriptMessage: 回调
- 销毁时需手动移除：removeScriptMessageHandlerForName:

⚠ iOS 与其他两端不同：Native 对象挂在 window.webkit.messageHandlers 下，而非直接挂在 window 上。

7.2 Android — addJavascriptInterface

```
// KRWebView.kt - init 中
webView.addJavascriptInterface(jsBridge,
    JSBridgeProtocol.NATIVE_HANDLER_NAME)
```

- 将 Kotlin 对象直接挂载到 window.KuiklyNativeHandler
- 标注 @JavascriptInterface 的方法暴露给 JS 调用
- 销毁时移除：removeJavascriptInterface()

7.3 OHOS — javascriptProxy

```
// KRWebView.ets - @Builder 中
Web({ src: '', controller: this.renderView.controller })
    .javascriptProxy({
        object: this.jsBridge.getJavaScriptProxyObject(),
        name: 'KuiklyNativeHandler',
        methodList: ['postMessage'],
        controller: this.controller
    })
```

- 声明式配置，Web 组件创建时生效
- methodList 限定 JS 只能调用指定方法
- 组件销毁时自动清理

对比表

维度	iOS	Android
挂载 API	addScriptMessageHandler	addJavascriptInterface
挂载位置	setupWebView()	init {}
JS 路径	window.webkit.messageHandlers.KuiklyNativeHandler	window.KuiklyNativeHandler
方法暴露控制	WKScriptMessageHandler 协议	@JavascriptInterface 注解
卸载方式	手动 removeScriptMessageHandlerForName:	手动 removeJavascriptInterface

8. Native 处理 JSON 消息的流程

三端都通过 两级处理：handleMessage → handleNativeCall。

8.1 第一级：handleMessage — 解析 JSON 并分发

Android：

```
private fun handleMessage(message: String) {
    try {
        val json =
            JSONObject(message) // 解析 JSON
        val type =
            json.optString(JSBridgeProtocol.KEY_TYPE) // 提取 type
        when (type) {
            JSBridgeProtocol.TYPE_CALL ->
                handleNativeCall(json) // type == "call"
            else ->
                onMessage(message) // 其他类型透传
        }
    } catch (e: Exception) {
```



```

        onMessage(message) //
        解析失败透传
    }
}

```

iOS :

```

- (void)handleMessage:(NSString *)message {
    NSData *data = [message
dataUsingEncoding:NSUTF8StringEncoding];
    NSDictionary *json = [NSJSONSerialization
JSONObjectWithData:data options:0 error:&error];
    // ... type == "call" → handleNativeCall:
}

```

OHOS :

```

private handleMessage(message: string): void {
    const json = JSON.parse(message) as Record<string, Object>;
    // ... type === 'call' → this.handleNativeCall(json)
}

```

8.2 第二级：handleNativeCall — 提取参数、查找 handler、执行

提取 callId / method / params



8.3 三端 JSON 解析方式对比

对比项	Android	iOS	OHOS
解析 API	JSONObject(message)	NSJSONSerialization	JSON.parse(message)
结果类型	JSONObject	NSDictionary	Record<string, Object>

对比项	Android	iOS	OHOS
线程处理	mainHandler.post {}	dispatch_async(main)	同步
handler 存储	ConcurrentHashMap	NSMutableDictionary	Map<string, Handler>

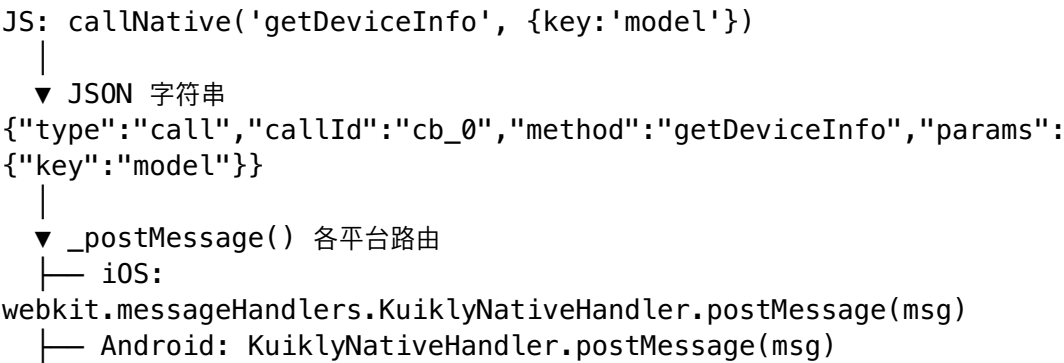
9. 消息路由：从 postMessage 到 handleMessage

以 Android 端为例，完整链路如下：

三步“接线”

步骤	动作	代码
① 装电话	把 Kotlin 对象安装到 JS 的 window 上	addJavascriptInterface(jsBridge, "KuiklyNativeHandler")
② 接线	告诉 JS “_postMessage 请拨打 KuiklyNativeHandler 的号码”	_postMessage = window.KuiklyNativeHandler.postMessage
③ 拨号	JS 业务代码发起调用	callNative('getDeviceInfo', {...})
④ 转接	Android 引擎通过反射找到 @JavascriptInterface 方法	引擎内部机制
⑤ 接听	Kotlin 的 postMessage 收到消息，转到主线程	mainHandler.post { handleMessage(message) }

完整调用序列图



```

[@JavascriptInterface]
└─ OHOS:    KuiklyNativeHandler.postMessage(msg)
[javascriptProxy]
|
▼ Native 解析 JSON → 执行 handler → 得到结果
|
▼ Native 调用 evaluateJavaScript / runJavaScript
window.KuiklyBridge._onNativeResponse('cb_0', {"model":"iPhone
15"}, null);
|
▼ JS Promise resolved
result = { model: "iPhone 15" }

```

10. Android @JavascriptInterface 反射机制详解

10.1 什么是反射

反射（Reflection）是 Java/Kotlin 的能力：在运行时根据字符串名称动态查找和调用类的方法，而不需要编译时硬编码调用关系。

```

// 正常调用（编译时确定）
jsBridge.postMessage("hello")

// 反射调用（运行时动态查找）
val method = jsBridge.javaClass.getMethod("postMessage",
    String::class.java)
method.invoke(jsBridge, "hello")

```

10.2 注册阶段

当调用 `addJavascriptInterface(jsBridge, "KuiklyNativeHandler")` 时，WebView 引擎内部：

1. 获取对象的 Class 信息
2. 反射扫描所有公开方法
3. 过滤：只保留带 `@JavascriptInterface` 注解的方法
4. 在 V8 引擎中创建 JS 代理对象 `window.KuiklyNativeHandler`
5. 为代理对象绑定同名 JS 函数 `postMessage()`
6. 保存映射关系：name → (对象实例, Method 对象)

伪代码：

```

void addJavascriptInterface(Object obj, String name) {
    Class<?> clazz = obj.getClass();
    Map<String, Method> exposedMethods = new HashMap<>();

    for (Method method : clazz.getMethods()) {
        // 只暴露带 @JavascriptInterface 注解的方法
        if (method.isAnnotationPresent(JavascriptInterface.class)) {

```

```

        exposedMethods.put(method.getName(), method);
    }
}

// 在 V8 引擎中创建 JS 代理对象
v8Engine.createJSProxy(name, exposedMethods);
interfaceMap.put(name, new BoundObject(obj, exposedMethods));
}

```

10.3 调用阶段

当 JS 执行 `window.KuiklyNativeHandler.postMessage(msg)` 时：

JS 调用 `window.KuiklyNativeHandler.postMessage(msg)`

- |
- ▼ V8 引擎拦截：这是一个 Native 代理对象的调用
- |
- ▼ 通过 JNI 桥接回到 Java/Kotlin 层
- |
- ▼ 根据 "KuiklyNativeHandler" 找到 jsBridge 对象
- |
- ▼ 根据 "postMessage" 找到 Method 对象
- |
- ▼ `method.invoke(jsBridge, msg)` ← 反射调用
- |
- ▼ 实际执行 `KRWebViewJSBridge.postMessage(msg)`
- |
- ▼ `mainHandler.post { handleMessage(msg) }`

10.4 @JavascriptInterface 注解的安全意义

Android 4.2 (API 17) 之前，`addJavascriptInterface` 会暴露对象的所有公开方法，包括从 `Object` 继承的 `getClass()`。恶意 JS 可以通过反射链执行任意系统命令：

```

// 恶意攻击代码 (Android 4.2 之前有效)
window.KuiklyNativeHandler.getClass()
    .forName('java.lang.Runtime')
    .getMethod('exec', ...)
    .invoke(null, 'rm -rf /')

```

Android 4.2+ 引入 `@JavascriptInterface` 注解作为白名单机制：只有标注了此注解的方法才会暴露给 JS，未标注的方法 JS 调用会得到 `undefined`。

10.5 为什么必须用反射

WebView 引擎是通用的系统组件，编译时不可能知道开发者会注册什么对象和方法。只能在运行时通过反射“发现”并调用开发者注册的类上的方法。

11. 完整调用链路总结

一次 JS → Native → JS 的完整数据流





设计亮点

1. **Kotlin Multiplatform 共享协议**：通过 JSBridgeProtocol.kt 统一定义消息格式、JS 脚本、工具方法，三端复用
2. **策略模式**：Bridge 核心逻辑统一，仅 _postMessage 一个方法因平台而异
3. **Promise 封装**：对 JS 开发者友好，支持 async/await
4. **超时保护**：30 秒超时机制防止 Native 不回调导致内存泄漏
5. **脚本预缓存**：避免每次页面加载时重复拼接字符串
6. **O(n) 字符串转义**：单次遍历 StringBuilder 替代多次 replace 链式调用，性能优化显著
7. **安全机制**：Android @JavascriptInterface 白名单、OHOS methodList 显式声明

文档生成日期：2026-03-18 基于 KuiklyWebview 项目源码分析